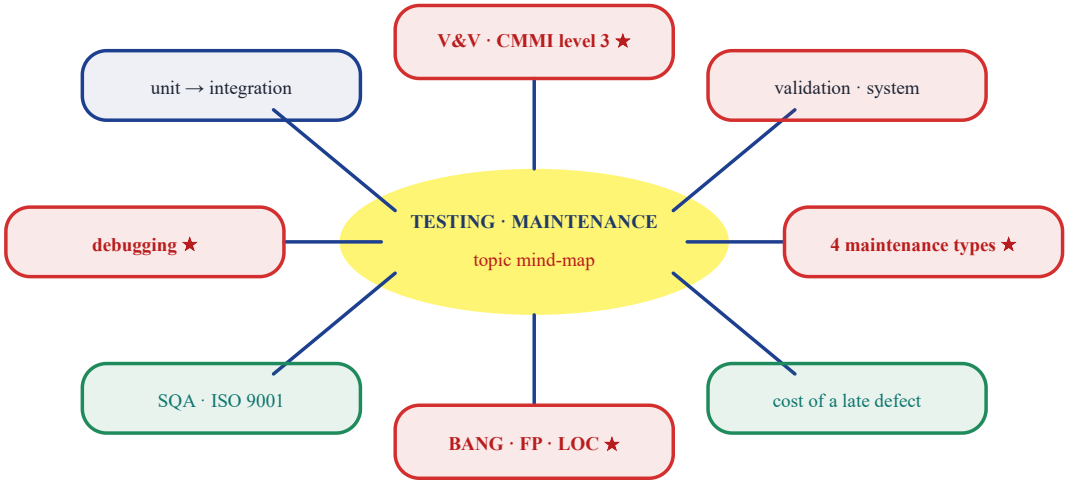
Testing, Maintenance, Quality & Metrics

Four levels · debugging · CAPPP · SQA · quality models & metrics: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

Software Testing: **white box and black box testing**, **basis path testing**, testing for a specialized environment; **software debugging techniques**; **software maintenance and upgrading**; **software quality assurance and quality models**; **software metrics**. Design-level test techniques (structural vs functional, cyclomatic complexity, basis path, EP / BVA) sit in *GRASP, DCD & Design-Level Testing*.

High-yield
① unit → integration → validation → system
② top-down = **stub**
③ CAPPP maintenance
④ 60–80 % is maintenance
⑤ earlier = cheaper fix

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Verification vs validation … p. 2
✓ The four testing levels … p. 2
✓ Integration strategies … p. 3
✓ Validation & system testing … p. 4
✓ Non-functional test types … p. 4
✓ Debugging … p. 5 · Maintenance CAPPP … p. 6
✓ Maintenance process & cost … p. 7
✓ SQA & standards … p. 8
✓ McCall, metrics families … p. 9
✓ Defect & reliability metrics … p. 10
✓ Night-before p. 11 · Practice p. 12

1 Verification vs Validation

The distinction: **Verification** asks **"are we building the product *right*?"** — is it being built to the **design**? **Validation** asks **"are we building the *right* product?"** — is it what the **user** wants? Verification is internal, done by the team; validation needs the **customer**.

Aspect	Verification	Validation
Tested against	the design / build specification	the **SRS** and user needs
Done by	developers: reviews, unit tests	testers **with the customer**

Where CMMI puts it: The **Verification** process area was introduced at **CMMI level 3 (Defined)** — **not** required at level 2. In CMMI 1.1 the area was called **Verification and Validation**. Levels: 1 Initial · 2 Repeated · 3 Defined · 4 Managed · 5 Optimizing.

Exam trap: Verification is against the *design*, validation against the *requirements*. Testing can show the **presence** of defects, never their absence — so a green validation suite does not "prove" the system right.

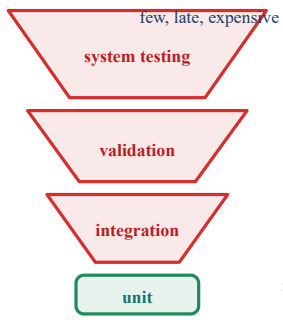
2 The Four Testing Levels, in Order

Level	The question it answers	Tested by
Unit testing	do the **parts** work right?	developers (JUnit, xUnit)
Integration testing	do the parts work **together**?	developers + testers
Validation testing	are we building the **right** product?	customer, at acceptance
System testing	does the **whole** system work?	testers, in a test environment

Unit: One module, class or method, in isolation — normally automated. Test-first (TDD): write the failing test before the method.

Integration: Catches the classic fault: every module passes alone, but the **interfaces disagree** on data format, order or timing.

System: The integrated whole run **as a product**: does it do what it promises, *and* how well? Non-functional types on the next sheet.



the test pyramid: broad at the bottom, narrow at the top

Memory trick: The four questions ask about **breadth**: parts → together → right thing → **whole**. Unit asks "does the engine work?", integration "do the wheels turn?", validation "is it *my* car?", system "does it drive on the road?" V-model: business req ↔ acceptance · system req ↔ system test · architecture ↔ integration · design ↔ unit.

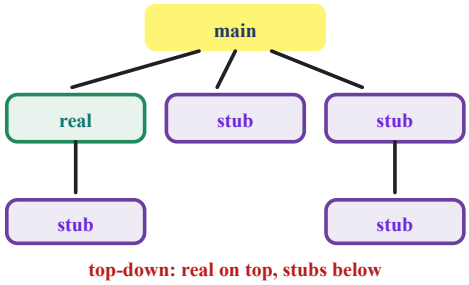
3 Integration Testing: Four Strategies

The problem: Modules tested in isolation all passed; assembled, they fail. Integration testing exposes the fault that lives **in the interfaces**. The strategy decides **in which order** modules are joined.

Strategy	Order of joining	Main weakness
Big Bang	all modules integrated **at once**	one failure, and the culprit is hard to isolate
Top-down	starts at the **main control module**, then downward	many stubs; the real lower modules are tested late
Bottom-up	starts at the **lowest leaf modules**, upward	the main control module is written **last**
Sandwich (hybrid)	top-down above, bottom-up below, meeting in the middle	more up-front work; broadest practical coverage

Stub: A **dummy replacement** for a module that is called but not yet written; it returns a canned value. **Top-down** uses it. `int add(int a,int b){ return 0; }`

Driver: A small **main routine written to call** the module under test and display the result. **Bottom-up** uses it. `main(){ printf("%d", sq(4)); }`



top-down integration — the arrow always points downward

Memory trick: The word tells you the stand-in. A **stub is called** → go **top**-down. A **driver calls** → go **bottom**-up. If you remember nothing else: top-down = stub, bottom-up = driver.

Exam trap: **Big Bang is not a level of testing** — it is one of the four joining *strategies*, and the least useful of them. In bottom-up testing the temporary piece is the **driver**, not the stub.

Exam trap: Sandwich (hybrid) is top-down for the upper layers and bottom-up for the lower ones, joined in the middle. Some books also list **threaded integration**: each module is integrated as soon as it is ready.

4 Validation & System Testing

Validation testing: Confirms the system does what the **SRS** says, with the customer in the loop. Its final stage is **acceptance testing**: **alpha** — on the developer's own site, staff as users; **beta** — on the customer's site, with **real users**. Cases are traced back to requirements in a traceability matrix.

System testing: The integrated system exercised **as a complete product**. It has two halves: **functional** testing (does it do what it promises?) and **non-functional** testing (how well, how fast, how safely?).

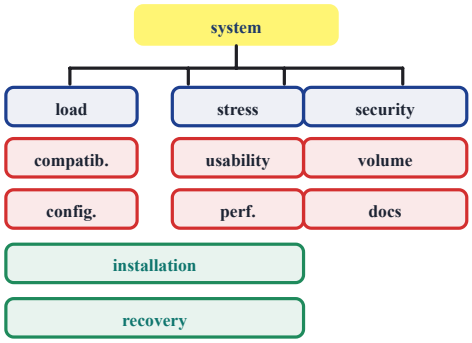
The seven non-functional / system testing types

Type	What it proves	Typical question
Load testing	behaviour under the **expected and peak** workload	response time at 500 users?
Stress testing	behaviour **beyond** capacity, up to breaking	at what load does it collapse?
Security testing	authentication, authorisation, data privacy	SQL injection? XSS?
Compatibility testing	works on the required browsers, OS, devices	renders in every browser?
Usability testing	a real user can learn and use it unaided	effort to learn?
Installation testing	installs, upgrades, uninstalls on a clean machine	setup succeeds on Windows 11?
Recovery testing	restored from failure within tolerance	how long to restore?

Exam trap: **Load** = the load you *expect*; **stress** = more than you can bear. Meeting the load target while failing under stress is still a passing system; passing stress but missing the load SLA is a failure.

Memory trick: Seven letters, seven types: **L**oad, **S**tress, **S**ecurity, **C**ompatibility, **U**sability, **I**nstallation, **R**ecovery. **Volume** and **configuration** testing are their near neighbours.

Also listed with system testing: **Performance** (speed, resource use) · **Volume** (very large or very small data sets) · **Configuration** (every supported setting) · **Documentation** · **Localization** (language, currency, dates).



functional testing answers "what"; these answer "how well"

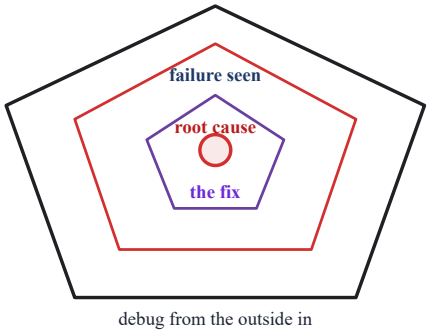
5 Debugging: Finding and Removing the Cause

Definition: **Debugging** is the process of locating, analysing and **removing the cause** of a defect — not of removing the *symptom*. In classic studies it takes **50–60 %** of a programmer's time.

The five steps, in order

1. **Reproduce** the failure deterministically — a bug you cannot reproduce is a bug you cannot close.
2. **Locate** the symptom: trace, logs, stack dump, breakpoints.
3. **Diagnose the root cause** — keep asking *why* until the answer stops blaming code.
4. **Fix** the cause, so the same class of bug cannot recur.
5. **Re-test**: the original case **plus a regression suite**, to prove nothing else broke.

Technique	How it works
Brute force	print statements, dumps, traces — let the program shout
Backtracking	start at the **point of failure**, work **backwards** to the last good state
Cause elimination	list candidate causes, test each; halve the data set
Program slicing	compute the slice of statements that can affect a point
Differential	compare a **known-good version** with the faulty one



debug from the outside in

never stop at the surface layer

Modern tools: Breakpoints, watch windows, single-step; logging; profilers; `git bisect` — backtracking through the commit history.

5b Debugging anti-patterns (Glass)

Anti-pattern	What it looks like	Right move
Mysterious change	the defect "moved" when you fixed something else	one change at a time; keep a record
Dynamic change	the fix works here and breaks there	regression-test the whole suite
Cut-and-paste	paste a corrected copy of a block somewhere else	find out why the first copy was wrong
Shotgun	change everything at random until the symptom goes	stop, revert, bisect
Dormant bug	intermittent — "works most of the time"	a shared-state race: instrument, do not hope
Assumption	"the compiler / library will do what I think"	check the contract, write the assertion

Exam trap: Debugging is **not** testing. **Testing** reveals that a defect exists; **debugging** finds and removes its cause. The defect has been in the code since it was written — it is the **latent** condition, and the failure is only the symptom that exposed it.

IT link: The chain error → defect → failure: the developer **misreads** the fee rule (error), ships the wrong rate (defect), and the statement is wrong (failure).

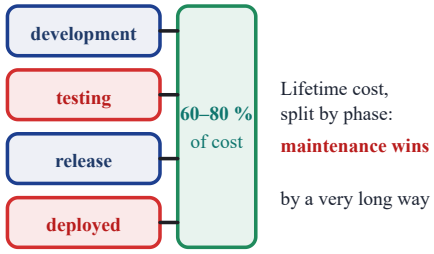
6 Software Maintenance: Why It Dominates

Definition: **Software maintenance** (IEEE Std 1219) is modification of a delivered system to **correct faults, improve performance or other attributes, or adapt it to a changed environment**. Delivery is not the end of the project — it is the start of its longest phase.

The number to remember: **60–80 %** of the total life-cycle cost of a software system is spent in maintenance; most surveys put it nearer **two thirds**. A system that is never maintained becomes obsolete; one maintained badly becomes expensive.

Why maintenance dominates

- the system stays in service for **years**; development lasts months;
- the environment moves on — **hardware, OS, database, third-party APIs, laws**;
- the original developers have **left**; nobody remembers the intent;
- real users find requirements that analysis **missed**;
- a **defect that escapes** costs 20–100× more to fix here than in requirements.



development is the small slice

7 The Four Types of Maintenance

Type	Purpose	Example
Corrective (remedial, repair)	fix faults found **after** delivery	an off-by-one in the payroll calculation; a tax rate hard-coded for last year
Adaptive (environmental)	make the software work in a **new environment**	porting an app from Windows to Linux; adopting a payment gateway's new API
Perfective (improvement)	add or improve **functionality or performance**	a new MIS report; a faster index; a friendlier screen
Preventive	restructure now to **prevent** future faults	refactoring a legacy module; deleting unreachable code; adding regression tests

Exam trap: **Adaptive is about the environment changing, not a new user requirement.** A new OS, database or law is **adaptive**; a report the user asked for is **perfective**; fixing a crash is **corrective**; rewriting a module so future change is safer is **preventive**.

Where the effort goes: Field studies (Lientz & Swanson) report **perfective ≈ half** of all maintenance effort, adaptive ≈ 25 %, corrective ≈ 20 %, preventive only a few per cent — the smallest slice, and the one that pays for itself.

8 The Maintenance Process

Inputs: A **change request** is logged, classified and costed before anything is touched, then passes **change control**: small fixes take a one-step approval, larger ones two-step.

1. **Receive & log** the change request / problem report
2. **Classify and estimate** — type (corrective, adaptive, …), effort, cost, risk
3. **Impact analysis** — which modules, interfaces and documents does this touch?
4. **Design & implement** the change; keep every change **versioned**
5. **Regression-test** the changed **and** the dependent modules
6. **Release**, update the documentation, close the request with a record

What makes maintenance expensive

- **Size and age** — more code, more interfaces, more ways to break.
- **Cruciality** — the core banking system cannot be down for a week.
- **Contractual duty** — a support period in the contract.
- **Documentation** — stale documents make each change a fresh investigation.
- **Staff turnover** — knowledge leaves with the developers.

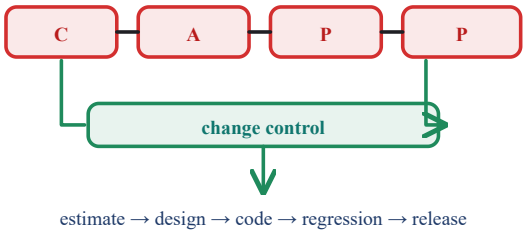
Lehman's Law of Continuing Change: (Lehman, 1972, refined 1980): **software must be changed or become obsolete**. In **E-type** systems complexity **increases** with every change unless maintenance is deliberate — hence preventive work.

Boehm's maintenance cost model: Annual effort **rises with the age** of the system: more users, data, interconnections, faults. A young system is cheap to keep; an old one is not.

Documentation is the precondition: Four documents must stay alive: **requirements**, **design**, **test cases** and the **user / operational manual** — the first three serve the maintainer, the last the operator. Maintenance without documentation is **archaeology**.

Memory trick: the four types: Corrective = it is **b**roken · **A**daptive = the world **a**round it moved · **P**erfective = it could be **p**rettier · **P**reventive = stop it **b**reaking later. CAPPP, in the order every syllabus prints it.

Exam trap: **Re-engineering** (rebuilding the system from the existing one) and **forward maintenance** are maintenance *activities*, not a fifth type — re-engineering is a perfective / preventive decision. The four types were classified by **IEEE** Std 1219, not by one author.



four request types, one controlled process

9 Software Quality Assurance

Definition: **Software Quality Assurance (SQA)** is the set of **planned and systematic activities** that provide confidence that quality requirements will be fulfilled. It is **not a phase**: it is a set of activities that **span the entire SDLC**, from requirements to maintenance.

Standards & procedures: Documented processes, guidelines and checklists the team must follow. Without them "quality" is only an opinion.

Reviews & inspections: Structured checking of artefacts at **requirements, design and code** — the cheapest defects in the life cycle. IEEE Std 1028 gives the review model.

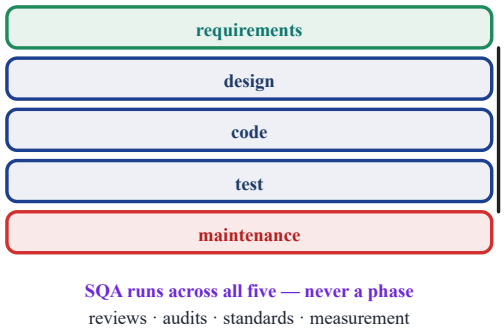
Audits: Check that the **process** is really being followed — "are the reviews happening and recorded?"

Testing at every level: Unit → integration → validation → system, plus defect tracking and closure from first report to final fix.

Configuration management: Control of code, documents and versions, so the right item is changed and the change is traceable.

Measurement & records: Defect density, review efficiency, coverage, cycle time — kept as **quality records**.

Standard / model	What it is
ISO/IEC 9001	the international **quality management system** requirements
ISO/IEC 12207	the **software life-cycle processes**; part 8 covers V&V
CMMI	process improvement model, **5 levels**; Verification at **level 3**
IEEE Std 730	the classic **software quality assurance** standard



SQA is a band, not a step

Exam trap: **SQA ≠ quality control.** QA is the *planned, preventive, process-wide* activity; QC is the **inspection** part — the tests themselves. "SQA is only testing" is the commonest wrong option.

10 McCall's Quality Factors

Category	Factors
Product operation — correctness of use	**Correctness · Reliability · Efficiency · Integrity · Usability**
Product revision — changeability	**Maintainability · Flexibility · Testability**
Product transition — portability	**Portability · Reusability · Interoperability**

Exam trap: McCall's model (McCall, Richards & Walters) is a **software product** quality model. Do not confuse it with **ISO/IEC 9126** (1991), whose six *characteristics* are functionality, reliability, usability, efficiency, maintainability and portability — later extended by ISO/IEC 25010 with **compatibility** and **security**. Eleven factors, six characteristics: do not mix the counts.

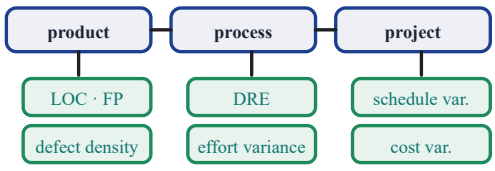
11 Software Metrics: Three Families

Family	Measured on	Examples
Product	the software product itself	size (LOC, function points), complexity, defect density, **MTBF**, MTTR
Process	the way the work was done	**DRE**, effort variance, review efficiency, defect-removal rate
Project	the project, planned vs delivered	schedule variance, cost variance, delivered size

Size, line-based: **LOC / KLOC** — simple and cheap, but style-dependent: the same job can be 200 or 2,000 LOC.

Size, function-based (FP): **Function Point Analysis** (Albrecht, IBM, 1979) counts the **functionality delivered**, so it is language-independent. Five components: **External Inputs, External Outputs, External Inquiries, Internal Logical Files, External Interface Files**. Unweighted EI 3, EO 4, EQ 3, ILF 7, EIF 5.

Complexity, line-based: **Cyclomatic V(G)** (see the GRASP notes) and **BANG**: **B**ranch + **A**ssignment + **N**umber of loops + **G**lobal variables. A **higher BANG means harder to test and maintain**.



phase metrics sit under all three

analysis → NOC, NOR · code → LOC, V(G)

testing → density, MTBF · maint. → MTTR

three families, and the phase each metric belongs to

12 Defect, Reliability & Availability Metrics

The formulas: **Defect density** = defects ÷ KLOC (or ÷ function points). **DRE** (Defect Removal Efficiency) = errors found **before** release ÷ total errors, × 100 %. **Availability** = MTBF ÷ (MTBF + MTTR).

Worked ✍️: MTBF = 1000 h, MTTR = 2 h ⇒ 1000 ÷ (1000 + 2) = 1000/1002 = **0.998 = 99.8 %**. Halving MTTR to 1 h gives 99.9 %: **doubling reliability is worth little while repair still takes hours**.

Reliability · Availability · Safety: **Reliability** = probability of failure-free operation over a stated period (MTBF). **Availability** = probability the system is operable when required. **Safety** = does no harm even *when* it fails — the concern for medical, aviation and nuclear systems.

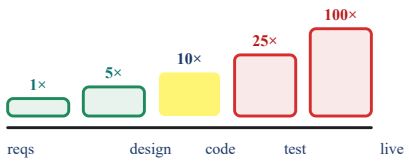
Phase-specific metrics

Phase	Metrics
Analysis & design	**NOC** classes · **NOR** requirements · **DIT** inheritance depth · **NOM** methods · use-case size
Source code	**LOC** · cyclomatic **V(G)** · **BANG** · coverage %
Testing	defect density · **DRE** · coverage · **MTBF**
Maintenance	**MTTR** · post-release defects · code churn

Exam trap: **Defect density is a product metric; DRE is a process metric.** DRE is always a *percentage* — "pre-release ÷ total" without × 100 % is incomplete. A **high DRE is good**; a high defect density is bad.

Cost of a defect found late

Found in	Requirements	Design	Coding	Testing	Production
Relative cost to fix	1×	5×	10×	15–25×	100×+



the later it is found, the steeper the bar

Boehm's field study in *Software Engineering Economics* (1981) is the classic source; ratios differ from study to study, so quote the **shape, not the digits as a law**. The mechanism is what matters: a requirements defect is **one line of a document**; a production defect is that line plus the code, the test, the release, the rollback and the customer's trust.

Night-Before-the-Exam Sheet

Fact sheet:

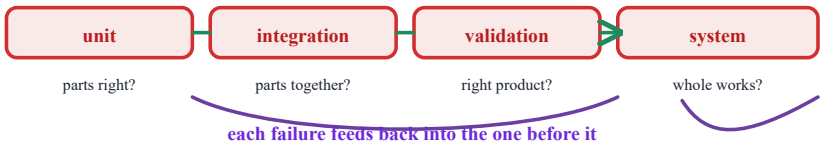
Unit → integration → validation → system	Verification = building it **right**; validation = **right** product
CMMI: Verification enters at **level 3**	Top-down uses **stubs**; bottom-up uses **drivers**
Big Bang · top-down · bottom-up · sandwich	Validation ends in acceptance: **alpha** then **beta**
Non-functional: load · stress · security	compatibility · usability · installation · recovery
Debug: reproduce → locate → diagnose → fix → re-test	Brute force · backtracking · cause elimination · slicing · differential
Glass: mysterious, dynamic, cut-paste, shotgun, dormant, assumption	Maintenance = **CAPPP**
60–80 % of life-cycle cost is maintenance	Lehman: change or become obsolete
SQA spans the SDLC; QA ≠ QC	ISO/IEC 9001 · 12207 · CMMI · IEEE 730
McCall: operation · revision · transition	FP: EI · EO · EQ · ILF · EIF
Defect density = defects ÷ KLOC	DRE = pre-release ÷ total × 100 %
Availability = MTBF ÷ (MTBF + MTTR)	Cost of a late defect: 1 · 5 · 10 · 25 · 100+

Top 14 one-liners for MCQs

1. White box = **glass box / structural**
2. Top-down integration → **stub**
3. Bottom-up integration → **driver**
4. New operating system → **adaptive** maintenance
5. Refactoring to avoid faults → **preventive**
6. SQA is **not** a phase; it spans the SDLC
7. QA (planned, preventive) ≠ **QC** (inspection)
8. Debugging = removing the **cause**
9. Work backwards from the failure = **backtracking**
10. Random changes until it works = **shotgun**
11. Load = expected load; **stress** = beyond capacity
12. McCall has **11** factors in **3** categories
13. FP measures **functionality**, not lines
14. Verification first required at CMMI **level 3**

Top traps: Verification is against the *design*, validation against the *requirements* · Big Bang is a strategy, not a level · stubs belong to top-down, drivers to bottom-up · a new OS is **adaptive**, a new report **perfective** · debugging ≠ testing · preventive maintenance is the *smallest* share of effort · SQA is not "the testing phase" · McCall's 11 factors ≠ ISO/IEC 9126's 6 characteristics · DRE is a percentage · a defect caught late costs far more.

All tricks together: Four questions of growing breadth: parts → together → right thing → whole · "stub is called, driver calls" · load is what you expect, stress is what you survive · a bug that moves is a shotgun · CAPPP in the order the syllabus prints it · 60–80 % explains a whole maintenance budget · earlier is cheaper, always.



the four levels — cheap at the bottom, expensive at the top

✓ Practice MCQs (exam pattern, 10 marks)

1. White-box testing is also known as:
(a) Behavioural testing (b) Acceptance testing
(c) Glass-box (structural) testing (d) Beta testing

2. In top-down integration, lower modules not yet written are replaced by:
(a) Stubs (b) Drivers
(c) Test oracles (d) Assertions

3. Keeping a system working after the organisation moves to a new operating system is:
(a) Corrective maintenance (b) Adaptive maintenance
(c) Perfective maintenance (d) Preventive maintenance

4. Preventive maintenance means:
(a) Porting an app to Linux (b) Adding a new MIS report
(c) Fixing a crash in production (d) Restructuring the code now to avoid future faults

5. Starting at the point of failure and working backwards is:
(a) Cause elimination (b) Program slicing
(c) Backtracking (d) Differential debugging

6. Changing unrelated parts of the program at random until the symptom disappears is:
(a) Shotgun debugging (b) Mysterious change
(c) Dynamic change (d) Dormant bug

7. In the BANG metric, the letter A counts:
(a) Branches (b) Assignments
(c) Arguments (d) Assertions

8. MTBF = 1000 h and MTTR = 2 h give an availability of about:
(a) 99.8 % (b) 50 %
(c) 99.0 % (d) 99.98 %

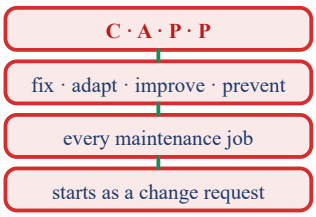
9. CMMI first makes **Verification** mandatory at level:
(a) 2 (Repeated) (b) 3 (Defined)
(c) 4 (Managed) (d) 5 (Optimizing)

10. Verification asks which question?
(a) Are we building the right product? (b) Is the system fast enough?
(c) Are we building the product right? (d) Is the system maintainable?

Answer key → 1-c · 2-a · 3-b · 4-d · 5-c · 6-a · 7-b · 8-a · 9-b · 10-c

Written question 1: Explain the **four levels of testing** with the question each answers, then the **four types of maintenance** with one example of each. 12 marks

1. **Unit** — do the parts work right? one module in isolation 2
2. **Integration** — do the parts work together? the interfaces 2
3. **Validation** — are we building the right product? against the SRS 2
4. **System** — does the whole system work? functional + non-functional 2
5. **Corrective** (fix faults) off-by-one in payroll · **Adaptive** (new environment) porting to a new OS 2
6. **Perfective** (add or improve) a new MIS report · **Preventive** (avoid faults) refactoring a legacy module 2



CAPPP, in the printed order

Written question 2: A core system has **MTBF = 1000 hours** and **MTTR = 2 hours**. Compute its availability, and explain with a table why a defect found in production can cost 100 times more than the same defect found in requirements. 10 marks

Written question 2 — answer: A = MTBF ÷ (MTBF + MTTR), and the defect-cost table from the metrics sheet.

1. A = MTBF ÷ (MTBF + MTTR) = 1000 ÷ 1002 = **0.998 ≈ 99.8 %** 3
2. Ratios: requirements **1**× · design **5**× · coding **10**× · testing **15–25**× · production **100**×+ 3
3. A requirements defect changes **one line of a document**; a production defect changes that line, the code, the test, the data and the customer's trust 4

Before the exam ✓

- ✓ Name the four levels with each one's question
- ✓ Say stub ↔ top-down and driver ↔ bottom-up without hesitating
- ✓ List the four maintenance types and test each with an example
- ✓ Write McCall's 11 factors under the right category
- ✓ Compute availability from MTBF and MTTR

Neighbours: white/black box, cyclomatic complexity, basis path, EP and BVA are in **GRASP, DCD & Design-Level Testing**; the waterfall, V-model and use cases in **Waterfall & UML**.